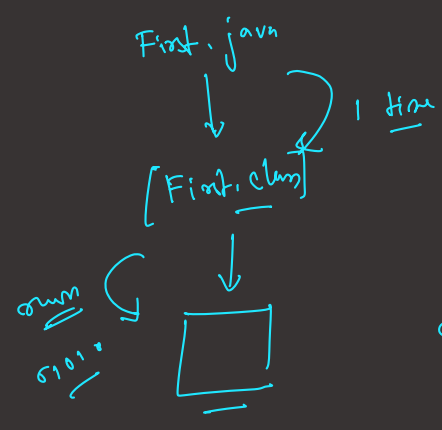
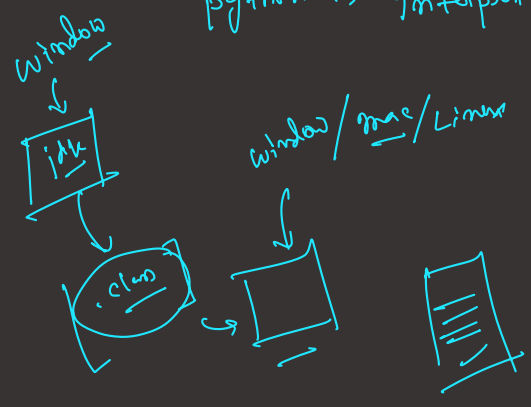
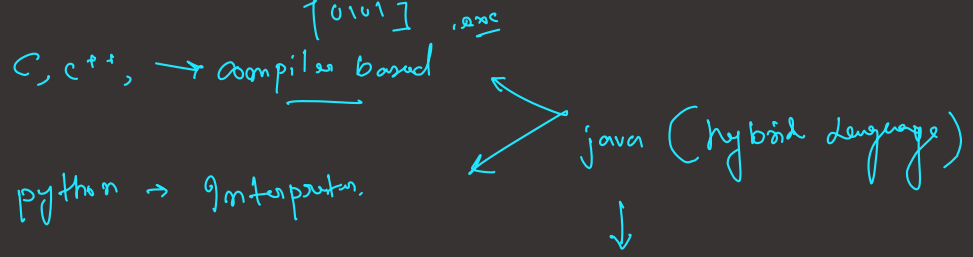


Day 1



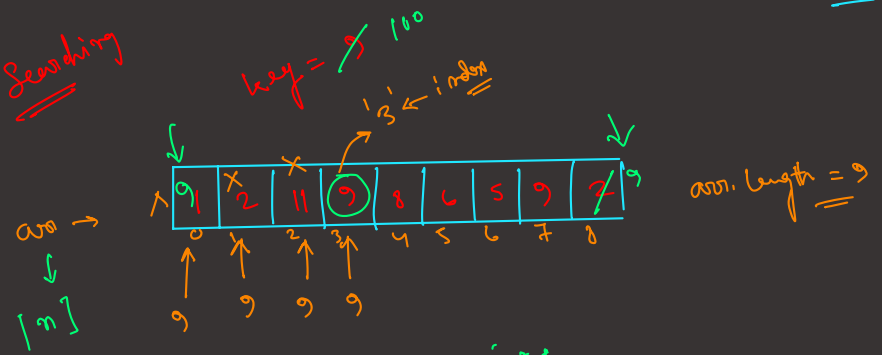
$O(n)$ mid [arr] can

key = first position $O(1)$

last index → $O(n)$

not in array

Searching



for (int i = 0 to arr.length) {

 searching;

}

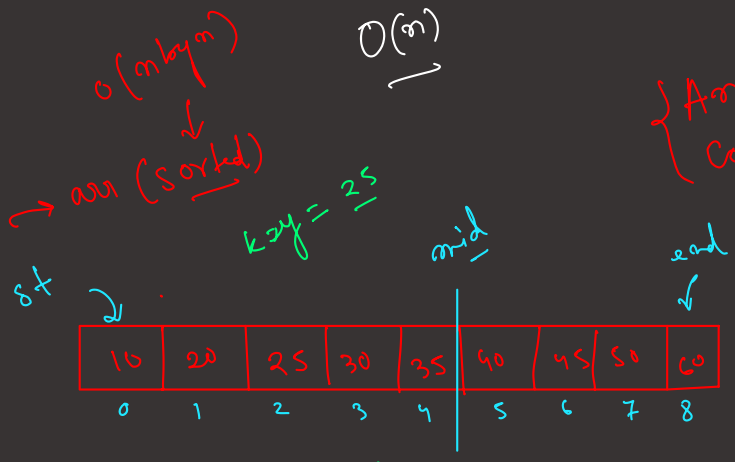
for (i = arr.length - 1 to 0) {

 searching;

}

$-2 < \underline{0} < 0$

Binary Search

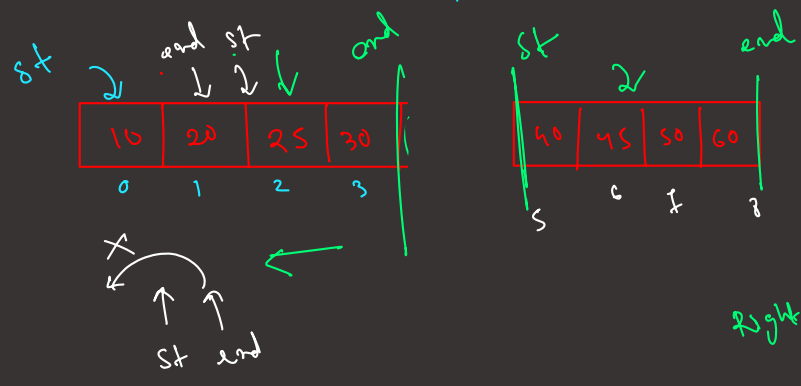


Arrays
Collections

$mid = \frac{st + end}{2}$

while (st ≤ end) = $\frac{0 + 8}{2} = 4$

(12)



right half → if (arr[mid] < key)

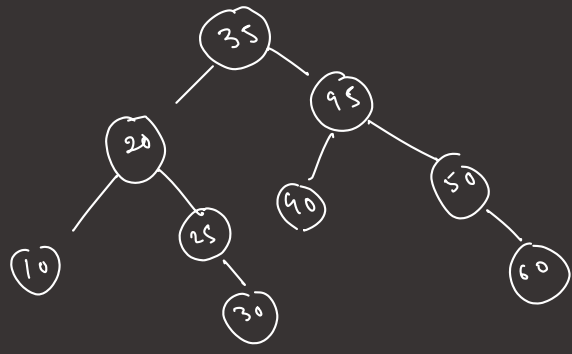
st = mid + 1;

$arr[mid] = 20 < key \rightarrow R.H$

$\{25, 30\}$
2 3

$mid = \frac{2+3}{2} = 2$
 $arr[mid] = 25 = key$

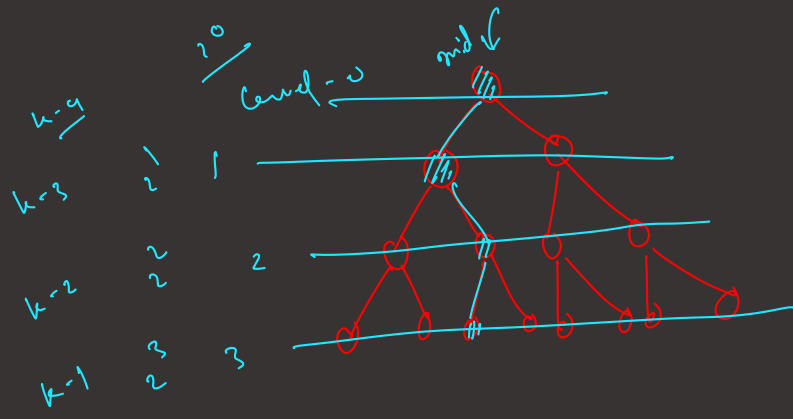
BST



left half : if $(arr[mid] > key)$
 $end = mid - 1$

← Searching

$O.P.C = O(\log n)$



$1 + 2 + 4 + 8 = 15$
 $\lfloor \log_2(15) \rfloor = 4$

total no of nodes
 $2^k - 1 = n$
↓
element

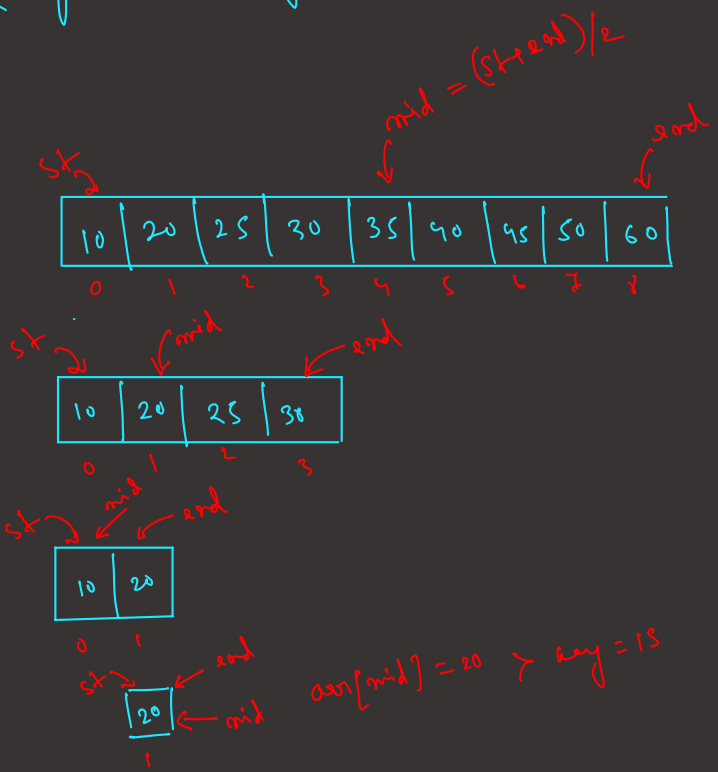
$2^k = n + 1$

$k = \log_2(n + 1)$

$= O(\log n)$

$1 < \log n < n < n \log n < n^2 < n^3 < n^n$ (recursion)

key = 15



H.W
key = 42

$st \leq end$ X

→ operation = -1

$st = 1$

← left half

$\{ end = mid - 1 = 1 - 1 = 0 \}$

$end = 0$

Find Max. or Min Subarray Sum

MaxSum = $-\infty$

brute force Approach
 $O.C = O(n^3)$
 MaxSum = $-\infty$

for (int i = 0 to n) {
 for (int j = i to n) {

sum = 0;

for (k = i to j) {

sum $\rightarrow$ (i, j)

}

MaxSum = max (MaxSum, sum);

}

}

return MaxSum;

for (i = 0 to n)
 for (j = i to n) {
 sum = 0

for (k = i to j) {

sum = sum + arr[k] (i to j)

MaxSum = max (MaxSum, sum)

$-\infty = \text{Integer.MIN_VALUE}$

Approach 2 : Prefix Sum Method, $O.C = O(n^2)$

arr =

1	-2	6	-1	3
---	----	---	----	---

if i = 0
 $\text{sum}(i, i) = \text{sum}(0, 0) = \text{prefix}[0]$

$\text{sum}(i, i) = \text{prefix}[i] - \text{prefix}[i-1]$

$\text{sum}(1, 3) = \text{prefix}[3] - \text{prefix}[0]$
 $= 9 - 1 = 8$

$\left[\text{sum}(2, 4) = \text{sum}(0, 4) - \text{sum}(0, 1) \right]$
 $\downarrow$
 $\text{prefix}[4]$

sum(0, 2) $\downarrow$
 sum(0, 4) = prefix[4]
 $\downarrow$
 prefix =

1	-1	5	4	7
0	1	2	3	4

 $\downarrow$
 $\text{prefix}[i] = \text{sum up to } i^{\text{th}} \text{ element of arr}$
 $= \text{sum}(0, i) \leftarrow \text{sum of element 0 to } i$

① calculate prefix Array.

② for (int i = 0 to n)
 for (j = i to n)

$\text{sum}(i, j) = \text{prefix}[j] - \text{prefix}[i-1]$

H.W find for min sub-array sum

Approach 3

Kadane's Algo $O(n) = O(n)$

-2	-3	4	-1	-2	1	5	-3
----	----	---	----	----	---	---	----

CS	0	0	4	3	1	2	7	4
MS	0	0	4	4	4	4	7	7

CS	-2	-3	4	3	1	2	7	4
MS	-2	-2	4	4	4	4	7	7

$$CS = \max(arr[i], CS + arr[i])$$

i=1 $CS = \max(-3, -2 + -3) = -3$

i=2 $CS = \max(4, -3 + 4) = 4$
 $CS = \max(4, 1) = 4$

i=3 $CS = \max(-1, -1 + 4) = 3$
 $CS = \max(-1, 3) = 3$

min + Max
 $-ve + +ve = +ve$
 $+ve + -ve = +ve \downarrow \uparrow +ve$

$$\left\{ \begin{array}{l} sum < 0 \\ CS = 0 \\ MS = \max(MS, CS) \end{array} \right\}$$

Qs Buy and sell stocks

prices = [7, 1, 5, 3, 6, 4]

buy at index 1, sell at index 4

profit = 6 - 1 = 5

profit = prices[i] - buying price

> 0 ↑

< 0 ↓

buy price = ∞, profit = 0

for (int i=0 to n)?

if (BP < prices[i])

profit = prices[i] - B.P. // day's profit.

max profit = max(max profit, today's profit);

i=3, 3 - 1 = 2
 today's profit = 2
 profit(2, 4)

BP = ∞, P = 0

i=0, Today's profit = 7 - ∞ = -∞
 buy price = 7

i=1, 1 - 7 = -6
 buy price = 1, P = 0

i=2, 5 - 1 = 4
 today's profit = 4
 max profit(0, 4) = 4

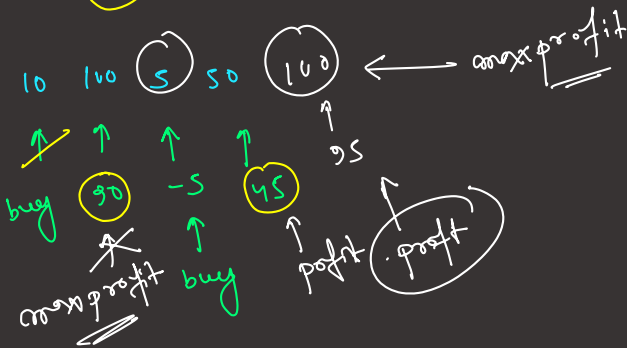
else {

buying price = prices[i];

}

}

90

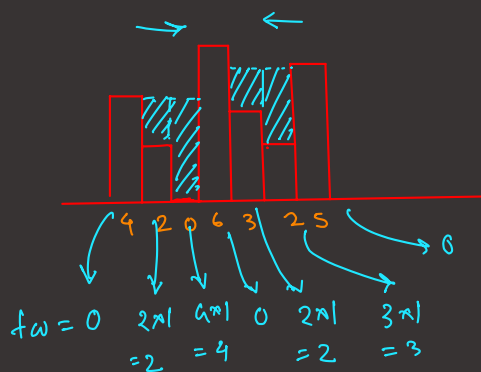


Day-3

Trapping Rainwater

Given n - non-negative integers representing an elevation map where the width of each bar is 1. compute how much water it can trap after raining

height = [4, 2, 0, 6, 3, 2, 5]

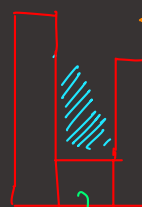


$\therefore$ total tw = 11

left boundary

right boundary

(i)



$w_i = \min(lb, rb) - \text{height}[i]$

tw = $w_i \times 1$

if $(n > 2 \text{ \& \& not acc \& not desc})$

return

return code

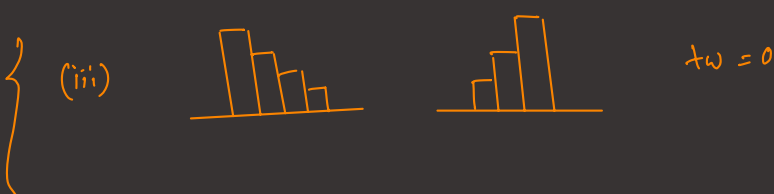
$O(n)$

acc trap = False

acc $\text{arr}[i-1] < \text{arr}[i]$

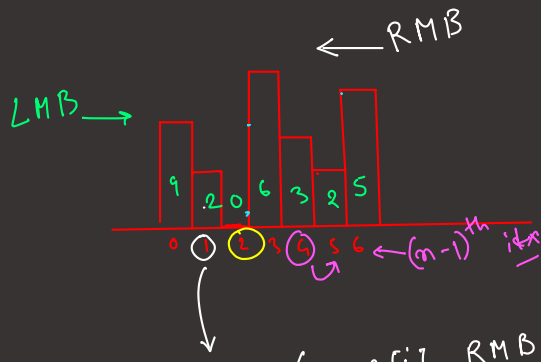
trap = true $\rightarrow$ Break

$n > 2$



if height[i] is sorted in acc or desc order

H.W



LMB =

4	4	4	6	6	6	6
0	1	2	3	4	5	6

 → Left Max Boundary

RMB =

6	6	6	6	5	5	5
0	1	2	3	4	5	6

 → Right Max Boundary

Trapped water at index i = $\underbrace{\min(LMB[i], RMB[i])}_{\text{Water level}} - \text{height}[i]$

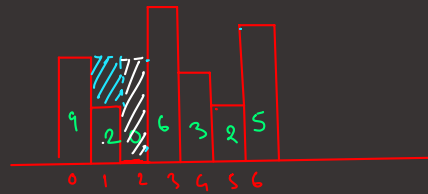
Potential water = $\sum tw$

$LMB[i] = \max(LMB[i-1], \text{height}[i])$ if $i \neq 0$

for $i=0$, $LMB[0] = \text{height}[0]$

$RMB[i] = \max(RMB[i+1], \text{height}[i])$ if $i \neq n-1$

if $i = n-1$, $RMB[i] = \text{height}[i]$



$$tw[0] = \min(LMB[0], RMB[0]) - \text{height}[0] \\ = \min(4, 6) - 4 = 4 - 4 = 0$$

$$tw[1] = \min(4, 6) - 2 = 4 - 2 = 2$$

$$tw[2] = \min(4, 6) - 0 = 4 - 0 = 4$$

⋮

Sorting

① Bubble Sort: large elements come to the end of array by swapping with the adjacent element.

A →

8	5	7	3	2
---	---	---	---	---

$i=0$
1st pass

8	5	5	5	5
5	8	7	7	7
7	7	8	3	3
3	3	3	8	2
2	2	2	2	8

Comp = 4
max swap = 4

result for pass 1
2nd pass $i=1$

5	5	5	5
7	7	3	3
3	3	7	2
2	2	2	7
8	8	8	8

2nd pass

Comp = 3
swap = 2
max swap = 3

$j=0$ to $n-2$

$j=0$ to $n-1$